Material de Estudo com Base nos Tópicos das Provas

Sistemas de Banco de Dados I

Docente: Elionai Magalhães

Table of Contents

Apresentação 2

Mapa dos conteúdos cobrados ..... **Erro! Indicador não definido.**

1. Sistema Gerenciador de Banco de Dados (SGBD) 2

2. Vantagens do SGBD em comparação aos arquivos tradicionais 3

3. Quando não usar um SGBD 4

4. Exemplos de SGBDs utilizados em aplicações web 5

5. Evolução histórica dos modelos de banco de dados 6

6. Modelo NoSQL: características gerais 8

7. Subtipos de bancos NoSQL 9

8. Modelo Entidade-Relacionamento: entidades fortes e fracas 10

9. Tipos de usuários de banco de dados 11

10. Arquitetura ANSI/SPARC: níveis interno, conceitual e externo 12

11. Linguagens de banco de dados: DDL, DML, DCL e TCL 13

12. Arquitetura cliente/servidor: duas camadas e três camadas 14

13. Restrições de integridade: entidade e referencial 15

14. Quadro-resumo para revisão rápida 16

15. Perguntas de revisão 17

16. Orientação para respostas discursivas 18

Referências básicas para aprofundamento 18

Apresentação

Este material foi elaborado a partir dos assuntos cobrados nas provas bimestrais de **Sistemas de Banco de Dados I**. O objetivo é reunir os principais tópicos em linguagem clara, com explicações, exemplos e pontos de atenção para orientar a revisão dos estudantes. O foco não é apenas memorizar alternativas, mas compreender os conceitos necessários para responder questões objetivas e discursivas sobre bancos de dados, SGBDs, modelos de dados, arquitetura, integridade e linguagens de banco de dados.

Como estudar: leia cada tópico, observe os exemplos e tente explicar o conceito com suas próprias palavras. Depois, responda às perguntas de revisão sem consultar o texto. Os itens marcados como “questões relacionadas” indicam em quais partes das provas aquele conteúdo apareceu.

1. Sistema Gerenciador de Banco de Dados (SGBD)

função principal do SGBD, vantagens em relação a arquivos e situações em que seu uso pode não ser indicado.

Um Sistema Gerenciador de Banco de Dados, conhecido como SGBD, é um software responsável por permitir a criação, organização, armazenamento, consulta, atualização, controle e proteção dos dados de uma organização. Ele funciona como uma camada intermediária entre os usuários ou aplicações e os dados armazenados.

A função principal de um SGBD é garantir que os dados sejam mantidos de forma eficiente, segura, consistente e acessível. Em vez de cada aplicação controlar seus próprios arquivos, o SGBD centraliza o gerenciamento dos dados e oferece mecanismos para reduzir erros, duplicações e inconsistências.

Principais funções de um SGBD:

- armazenar e organizar dados de maneira estruturada;
- permitir consultas, inserções, alterações e exclusões de dados;
- controlar o acesso dos usuários e proteger informações sensíveis;
- garantir integridade, consistência, recuperação e disponibilidade dos dados;
- apoiar o uso simultâneo por vários usuários ou sistemas.

Exemplo: em uma faculdade, um SGBD pode armazenar dados de alunos, professores, disciplinas, turmas, notas e matrículas. Quando um aluno consulta suas notas no portal acadêmico, a aplicação solicita ao SGBD os dados correspondentes e apresenta o resultado ao usuário.

Ponto de atenção: SGBD não é a mesma coisa que banco de dados. O banco de dados é o conjunto organizado de dados. O SGBD é o software que gerencia esse conjunto, como MySQL, PostgreSQL, Oracle ou SQL Server.

2. Vantagens do SGBD em comparação aos arquivos tradicionais

Vantagens do SGBD e comparação com sistemas baseados apenas em arquivos.

Antes da popularização dos SGBDs, muitas aplicações armazenavam informações em arquivos isolados, como arquivos de texto, planilhas ou arquivos binários próprios. Essa abordagem pode funcionar em sistemas pequenos, mas apresenta limitações quando os dados crescem, precisam ser compartilhados ou exigem controle rigoroso.

A abordagem com SGBD reduz a redundância, melhora a consistência, facilita a segurança, permite controle de acesso e oferece suporte ao acesso simultâneo. Além

disso, um SGBD possui mecanismos de backup, recuperação, integridade e transações, que aumentam a confiabilidade do sistema.

Vantagens importantes:

- **redução de redundância:** evita que a mesma informação seja repetida em vários arquivos;
- **consistência:** ajuda a impedir que o mesmo dado apareça com valores diferentes em locais distintos;
- **segurança:** permite definir permissões por usuário, perfil ou função;
- **integridade:** aplica regras para impedir dados inválidos;
- **concorrência:** permite que vários usuários acessem o banco ao mesmo tempo;
- **recuperação:** oferece mecanismos de backup e restauração em caso de falhas.

Exemplo: em um sistema baseado apenas em arquivos, o CPF de um aluno poderia aparecer em vários documentos diferentes. Se fosse necessário corrigir esse CPF, o responsável teria que localizar todos os arquivos. Em um SGBD, essa informação pode ser armazenada uma única vez em uma tabela de alunos, facilitando a manutenção.

Ponto de atenção: arquivos simples apenas armazenam dados. O SGBD, além de armazenar, controla acesso, regras, consistência, relacionamentos, consultas e recuperação.

3. Quando não usar um SGBD

Situações em que sistemas simples de armazenamento podem ser mais adequados.

Apesar das vantagens, nem sempre o uso de um SGBD é necessário. Em alguns casos, o custo de implantação, configuração, administração e manutenção pode ser maior do que o benefício obtido. Sistemas muito pequenos, temporários ou com dados simples podem usar arquivos comuns de forma satisfatória.

Um SGBD também pode ser desnecessário quando não há necessidade de múltiplos usuários, consultas complexas, controle transacional, segurança avançada ou relacionamento entre grandes volumes de dados. Nesses casos, arquivos CSV, JSON, TXT ou planilhas podem atender à necessidade.

Situações em que arquivos simples podem ser suficientes:

- aplicações muito simples, com poucos dados e sem crescimento previsto;
- protótipos rápidos ou atividades temporárias;
- sistemas de uso individual, sem concorrência de usuários;
- dados sem necessidade de consultas complexas ou regras de integridade;
- ambientes com recursos computacionais muito limitados.

Exemplo: um pequeno programa local que registra uma lista de tarefas pessoais pode armazenar os dados em um arquivo JSON. Usar um SGBD completo nesse caso pode ser exagerado, porque não há grande volume de dados, vários usuários simultâneos ou regras complexas de integridade.

Ponto de atenção: não usar SGBD não significa falta de organização. Significa escolher uma solução proporcional ao tamanho, ao risco e à complexidade do problema.

4. Exemplos de SGBDs utilizados em aplicações web

SGBDs open source e uso em sistemas online.

Aplicações web geralmente precisam armazenar usuários, senhas, permissões, produtos, pedidos, pagamentos, mensagens, logs e outros tipos de informação. Para isso, é comum o uso de SGBDs relacionais ou não relacionais.

Entre os SGBDs relacionais mais conhecidos estão MySQL, PostgreSQL, Oracle, Microsoft SQL Server e IBM Db2. O MySQL se destaca por ser open source, possuir ampla comunidade, ser muito usado em sistemas web e apresentar bom desempenho em muitas aplicações tradicionais. O PostgreSQL também é open source e é reconhecido por sua robustez e recursos avançados.

Exemplos:

- **MySQL:** muito usado em aplicações web, sites, sistemas administrativos e plataformas com PHP, Java, Python ou Node.js;
- **PostgreSQL:** usado em sistemas que exigem recursos avançados, integridade forte e consultas complexas;
- **Oracle:** comum em grandes corporações e sistemas críticos;
- **SQL Server:** muito utilizado em ambientes Microsoft;
- **IBM Db2:** presente em ambientes corporativos e legados de grande porte.

Exemplo: um sistema de cursos online pode usar MySQL para armazenar alunos, cursos, inscrições, avaliações e certificados. Uma aplicação corporativa com consultas analíticas mais complexas poderia usar PostgreSQL.

Ponto de atenção: open source significa que o software possui código aberto e pode ser usado, estudado e adaptado conforme sua licença. Isso não significa automaticamente que todos os serviços associados serão gratuitos.

5. Evolução histórica dos modelos de banco de dados

bancos em rede e evolução dos bancos de dados desde os anos 1960.

Os bancos de dados evoluíram conforme as necessidades dos sistemas de informação. Os primeiros modelos, como o hierárquico e o em rede, tinham estruturas mais próximas do armazenamento físico dos dados. Depois, o modelo relacional passou a organizar dados em tabelas, facilitando consultas e relacionamentos por meio de chaves.

Mais recentemente, surgiram bancos NoSQL e distribuídos para lidar com grandes volumes de dados, dados semiestruturados, alta escalabilidade e aplicações modernas, como redes sociais, comércio eletrônico, sistemas em nuvem e análise de dados.

Principais modelos:

- **modelo hierárquico:** organiza dados em estrutura de árvore, com relações pai-filho;
- **modelo em rede:** permite relações mais complexas entre registros, inclusive muitos-para-muitos, usando conexões ou ponteiros;
- **modelo relacional:** organiza dados em tabelas relacionadas por chaves primárias e estrangeiras;
- **modelos NoSQL:** priorizam flexibilidade, escalabilidade e diferentes formatos de armazenamento;
- **bancos distribuídos:** distribuem dados entre vários servidores para melhorar disponibilidade e escalabilidade.

Exemplo: em uma biblioteca, um livro pode ter vários autores e um autor pode escrever vários livros. Esse tipo de relacionamento muitos-para-muitos é um exemplo clássico de necessidade de representar conexões mais complexas entre registros, algo que o modelo em rede buscava atender.

Ponto de atenção: a evolução dos bancos de dados não eliminou os modelos anteriores em todos os contextos. Ela ampliou as alternativas. Hoje, a escolha depende do tipo de aplicação, volume de dados, necessidade de consistência, desempenho e escalabilidade.

6. Modelo NoSQL: características gerais

Características do NoSQL e seus subtipos.

NoSQL é um conjunto de modelos de banco de dados criado para atender cenários em que o modelo relacional tradicional pode não ser suficiente ou pode se tornar pouco flexível. Bancos NoSQL são comuns em aplicações com grande volume de dados, dados distribuídos, estruturas variáveis e necessidade de escalabilidade horizontal.

Diferentemente dos bancos relacionais, que normalmente usam tabelas com esquema bem definido, muitos bancos NoSQL aceitam estruturas mais flexíveis, como documentos, pares chave-valor, famílias de colunas ou grafos. Eles podem trabalhar com dados estruturados, semiestruturados e não estruturados.

Características gerais:

- modelo flexível de dados;
- escalabilidade horizontal, com distribuição em vários servidores;
- adequação a dados estruturados, semiestruturados e não estruturados;
- uso frequente em aplicações web, Big Data, redes sociais e sistemas de alta demanda;
- subtipos principais: chave-valor, documentos, colunar, grafos e séries temporais.

Exemplo: uma rede social pode usar um banco orientado a grafos para representar conexões entre pessoas. Um sistema de cache pode usar chave-valor para localizar rapidamente o conteúdo associado a uma chave. Um catálogo de produtos pode usar documentos para armazenar produtos com atributos diferentes.

Ponto de atenção: NoSQL não significa que SQL nunca seja usado. O termo indica que o modelo não segue exclusivamente a estrutura relacional tradicional.

Algumas soluções NoSQL oferecem linguagens de consulta próprias ou até recursos parecidos com SQL.

7. Subtipos de bancos NoSQL

Pares chave-valor e reconhecimento dos subtipos NoSQL.

Os bancos NoSQL possuem diferentes subtipos, cada um adequado a determinados problemas. Para estudar, é importante identificar como os dados são armazenados e em que situação cada tipo costuma ser utilizado.

O subtipo chave-valor armazena dados em pares formados por uma chave única e um valor associado. Ele costuma oferecer acesso muito rápido, mas geralmente não é o mais indicado para consultas complexas.

Subtipos principais:

- **chave-valor:** armazena dados como chave e valor. Exemplo: `id_usuario_15 -> dados do usuário`;
- **documentos:** armazena registros como documentos, geralmente em JSON ou BSON;
- **colunar ou família de colunas:** organiza dados por colunas ou grupos de colunas;
- **grafos:** representa entidades como nós e relacionamentos como arestas;
- **séries temporais:** otimizado para dados associados ao tempo.

Exemplo: em um sistema de autenticação, pode-se guardar a sessão de um usuário em um banco chave-valor: a chave é o token da sessão e o valor contém informações temporárias do usuário logado.

Ponto de atenção: para a prova, associe “pares chave-valor” diretamente ao subtipo chave-valor. A descrição geralmente menciona consultas muito rápidas e estrutura simples.

8. Modelo Entidade-Relacionamento: entidades fortes e fracas

Diferença entre entidade forte e entidade fraca.

O Modelo Entidade-Relacionamento, ou MER, é usado na modelagem conceitual de bancos de dados. Ele representa entidades, atributos e relacionamentos antes da implementação física no SGBD.

Uma entidade forte possui existência independente e normalmente tem uma chave própria que permite identificar seus registros. Uma entidade fraca depende de outra entidade para existir e para ser identificada. Geralmente, sua identificação combina uma chave parcial com a chave da entidade forte da qual depende.

Conceitos principais:

- **entidade forte:** existe independentemente e possui chave própria;
- **entidade fraca:** depende de uma entidade forte para existir;
- **chave parcial:** atributo que ajuda a identificar a entidade fraca, mas não é suficiente sozinho;
- **relacionamento identificador:** relacionamento que vincula a entidade fraca à entidade forte.

Exemplo: Pedido pode ser uma entidade forte, pois possui número próprio e existe como registro principal. ItemDePedido pode ser uma entidade fraca, pois depende de um Pedido. A identificação completa pode ser formada por número do pedido + número do item.

Ponto de atenção: a entidade fraca não é “menos importante”. Ela apenas depende de outra entidade para sua identificação e existência no modelo.

9. Tipos de usuários de banco de dados

Usuário final casual e usuário final sofisticado.

Diferentes usuários interagem com o banco de dados de maneiras diferentes. Alguns utilizam telas prontas de sistemas, enquanto outros formulam consultas, analisam dados ou desenvolvem soluções mais complexas.

O usuário final casual acessa o banco de dados de forma eventual, geralmente por meio de consultas simples ou relatórios já disponíveis. Já o usuário final sofisticado possui maior conhecimento técnico ou analítico e consegue formular consultas mais complexas, usar ferramentas de análise ou explorar os dados de forma mais independente.

Tipos de usuários:

- **usuário final casual:** acessa o sistema ocasionalmente para consultar informações específicas;
- **usuário final sofisticado:** usa ferramentas avançadas, cria consultas e analisa dados com maior autonomia;
- **programador de aplicações:** desenvolve sistemas que acessam o banco;
- **administrador de banco de dados:** gerencia segurança, desempenho, backup, usuários e estrutura do banco.

Exemplo: um coordenador que acessa o sistema para consultar a lista de alunos matriculados é um usuário casual. Um analista que cria consultas SQL para cruzar dados de matrícula, evasão e desempenho acadêmico é um usuário sofisticado.

Ponto de atenção: a diferença principal está no grau de autonomia e complexidade de uso. O casual usa funcionalidades prontas; o sofisticado explora os dados de forma mais avançada.

10. Arquitetura ANSI/SPARC: níveis interno, conceitual e externo

Arquitetura de três esquemas e independência de dados.

A arquitetura ANSI/SPARC propõe a separação do banco de dados em três níveis: interno, conceitual e externo. Essa separação ajuda a reduzir a dependência entre a forma como os dados são armazenados fisicamente, como são estruturados logicamente e como são apresentados aos usuários.

O nível interno descreve como os dados são armazenados no meio físico, incluindo arquivos, índices e estruturas de acesso. O nível conceitual descreve a estrutura lógica global do banco, como entidades, tabelas, atributos, relacionamentos e restrições. O nível externo corresponde às visões específicas de usuários ou grupos de usuários.

Resumo dos níveis:

- **nível interno:** armazenamento físico e estruturas de acesso;
- **nível conceitual:** visão lógica global do banco de dados;
- **nível externo:** visões específicas para usuários, setores ou aplicações;
- **independência física:** mudanças no armazenamento não devem afetar o modelo lógico;
- **independência lógica:** mudanças no modelo conceitual não devem obrigatoriamente afetar todas as visões externas.

Exemplo: o setor financeiro pode visualizar dados de pagamento, enquanto a secretaria acadêmica visualiza dados de matrícula. Ambos usam o mesmo banco, mas têm visões externas diferentes. Se o administrador criar um novo índice para melhorar o desempenho, essa mudança ocorre no nível interno e não precisa alterar as telas dos usuários.

Ponto de atenção: a independência de dados é uma das grandes vantagens dessa arquitetura, pois permite alterar um nível sem causar impacto direto e desnecessário nos demais.

11. Linguagens de banco de dados: DDL, DML, DCL e TCL

Categorias de linguagens oferecidas por um SGBD.

Um SGBD oferece linguagens ou subconjuntos de comandos para definir, manipular, controlar e gerenciar transações no banco de dados. Em muitas disciplinas introdutórias, as três categorias mais destacadas são DDL, DML e DCL. Também é comum mencionar a TCL como categoria complementar ligada ao controle de transações.

A DDL é usada para definir a estrutura do banco. A DML é usada para manipular os dados. A DCL é usada para controlar permissões. A TCL é usada para confirmar ou desfazer transações.

Principais categorias:

- **DDL - Data Definition Language:** cria, altera e exclui estruturas. Exemplos: `CREATE`, `ALTER`, `DROP`;
- **DML - Data Manipulation Language:** consulta e modifica dados. Exemplos: `SELECT`, `INSERT`, `UPDATE`, `DELETE`;
- **DCL - Data Control Language:** controla permissões e acesso. Exemplos: `GRANT`, `REVOKE`;

- **TCL - Transaction Control Language:** controla transações. Exemplos: `COMMIT`, `ROLLBACK`, `SAVEPOINT`.

Exemplo: `CREATE TABLE Aluno` cria a estrutura da tabela. `INSERT INTO Aluno` adiciona um aluno. `SELECT * FROM Aluno` consulta os registros. `GRANT SELECT ON Aluno` permite que um usuário consulte a tabela. `COMMIT` confirma uma transação realizada.

Ponto de atenção: em provas discursivas, explique a função de cada categoria e apresente pelo menos um comando de exemplo. Isso demonstra domínio conceitual e aplicação prática.

12. Arquitetura cliente/servidor: duas camadas e três camadas

Comparação entre arquitetura de duas camadas e de três camadas.

Na arquitetura cliente/servidor, as aplicações são organizadas em camadas. Essa divisão ajuda a separar responsabilidades e melhora manutenção, segurança e escalabilidade. As arquiteturas de duas e três camadas são bastante comuns em sistemas que acessam bancos de dados.

Na arquitetura de duas camadas, o cliente se comunica diretamente com o servidor de banco de dados. Parte da lógica da aplicação pode ficar no cliente, que envia comandos diretamente ao SGBD. Na arquitetura de três camadas, existe uma camada intermediária, geralmente chamada de servidor de aplicação, responsável por regras de negócio, validações e comunicação com o banco.

Comparação:

- **duas camadas:** cliente + servidor de banco de dados;
- **três camadas:** cliente + servidor de aplicação + servidor de banco de dados;

- duas camadas pode ser mais simples, mas tende a ter menor controle em sistemas grandes;
- três camadas melhora segurança, manutenção e escalabilidade;
- em três camadas, o cliente não precisa acessar diretamente o banco.

Exemplo: em um sistema desktop simples, o programa instalado no computador do usuário pode conectar diretamente ao banco de dados. Isso é duas camadas. Em um sistema web, o navegador acessa uma aplicação no servidor, e essa aplicação consulta o banco. Isso é três camadas.

Ponto de atenção: a camada intermediária é importante porque concentra regras de negócio, autenticação, validação e segurança, evitando que cada cliente acesse diretamente o banco de dados.

13. Restrições de integridade: entidade e referencial

Integridade de entidade e integridade referencial.

No modelo relacional, as restrições de integridade garantem que os dados permaneçam corretos, consistentes e coerentes. Duas restrições fundamentais são a integridade de entidade e a integridade referencial.

A integridade de entidade determina que toda tabela deve possuir uma forma de identificar unicamente seus registros, geralmente por meio de uma chave primária. Essa chave não pode ser nula, pois um registro sem identificação compromete a organização da tabela. A integridade referencial determina que uma chave estrangeira deve apontar para um registro existente na tabela relacionada, evitando relacionamentos inválidos.

Conceitos principais:

- **integridade de entidade:** a chave primária identifica cada registro e não pode ser nula;

- **integridade referencial:** a chave estrangeira deve fazer referência a uma chave existente em outra tabela;
- **chave primária:** identifica unicamente um registro;
- **chave estrangeira:** cria vínculo entre tabelas.

Exemplo: na tabela Aluno, o campo matrícula pode ser chave primária e não pode ficar vazio. Isso é integridade de entidade. Na tabela Matrícula, o campo id_aluno pode ser chave estrangeira apontando para Aluno. Não deve ser possível registrar uma matrícula para um aluno inexistente. Isso é integridade referencial.

Ponto de atenção: a integridade de entidade cuida da identificação do próprio registro. A integridade referencial cuida da validade do relacionamento entre registros de tabelas diferentes.

14. Quadro-resumo para revisão rápida

Conceito	Ideia principal	Exemplo
SGBD	Software que gerencia dados	MySQL gerenciando dados de alunos
Banco de dados	Conjunto organizado de dados	Tabelas Aluno, Curso e Matrícula
Chave primária	Identifica um registro de forma única	matrícula do aluno
Chave estrangeira	Relaciona uma tabela a outra	id_curso em uma tabela de matrícula
NoSQL chave-valor	Dados em pares chave e valor	token_sessao -> dados do usuário
Entidade forte	Existe independentemente	Cliente ou Pedido
Entidade fraca	Depende de outra entidade	ItemDePedido
Duas camadas	Cliente acessa diretamente o banco	Aplicação desktop + banco
Três camadas	Cliente acessa aplicação, que acessa banco	Navegador + servidor web + banco

DDL	Define estruturas	CREATE TABLE
DML	Manipula dados	SELECT, INSERT, UPDATE
DCL	Controla permissões	GRANT, REVOKE

15. Perguntas de revisão

1. Explique, com suas próprias palavras, qual é a função principal de um SGBD.
2. Cite três vantagens do SGBD em relação ao uso de arquivos tradicionais.
3. Em quais situações pode ser mais adequado usar arquivos simples em vez de um SGBD?
4. Por que o MySQL é frequentemente usado em aplicações web?
5. Diferencie banco relacional e banco NoSQL.
6. O que caracteriza um banco NoSQL do tipo chave-valor?
7. Dê um exemplo de entidade forte e entidade fraca em um sistema de vendas.
8. Explique os níveis interno, conceitual e externo da arquitetura ANSI/SPARC.
9. Diferencie usuário final casual e usuário final sofisticado.
10. Compare arquitetura cliente/servidor de duas camadas e de três camadas.
11. Explique a diferença entre DDL, DML e DCL, com exemplos de comandos.
12. Diferencie integridade de entidade e integridade referencial.

16. Orientação para respostas discursivas

Nas questões discursivas, uma boa resposta deve apresentar conceito, explicação e exemplo. O aluno deve evitar respostas com apenas uma frase curta, pois isso pode não demonstrar domínio do conteúdo.

Uma estrutura simples para responder é: primeiro definir o conceito; depois explicar sua importância; por fim apresentar um exemplo prático. Essa organização torna a resposta mais clara e completa.

Modelo de resposta: a integridade referencial é uma regra do modelo relacional que garante que uma chave estrangeira aponte para um registro existente em outra tabela. Ela evita inconsistências nos relacionamentos. Por exemplo, uma matrícula não deve ser cadastrada para um aluno que não exista na tabela Aluno.

Referências básicas para aprofundamento

DATE, C. J. *Introdução a Sistemas de Bancos de Dados*. 8. ed. Rio de Janeiro: Elsevier, 2004.

ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de Banco de Dados*. 7. ed. São Paulo: Pearson, 2018.

KORTH, Henry F.; SILBERSCHATZ, Abraham; SUDARSHAN, S. *Sistema de Banco de Dados*. 6. ed. Rio de Janeiro: Elsevier, 2012.

ROB, Peter; CORONEL, Carlos. *Sistemas de Banco de Dados: Projeto, Implementação e Gerenciamento*. São Paulo: Cengage Learning, 2011.